

Technical Specifications

Version: 1.0

Purpose

This document defines the technical specifications and implementation constraints of Trading Platform Pro and its primary application, the Trading Cockpit.

The specifications translate the Product Vision and Product Roadmap into technical requirements.

Technical capabilities shall support concrete Trading Cockpit workflows and shall not be generalized prematurely without an identified product requirement.

Primary Application

The primary application is the Trading Cockpit.

The Trading Cockpit is a professional desktop application for:

- market observation
- instrument selection
- trading candidate evaluation
- portfolio context
- risk context
- decision support
- order preparation and execution
- position monitoring
- trading review

The application shall support incremental vertical product slices.

Technology Stack

Primary technologies:

- Python 3.13
- PySide6
- SQLite
- SQLAlchemy
- Dependency Injector
- AsyncIO
- Pytest
- Ruff
- GitHub Actions
- Pandoc

Technology changes require explicit technical evaluation and documentation where they affect architecture or product capabilities.

Target Platforms

Primary platform:

- Windows

Future platforms:

- Linux
- macOS

Windows is the reference platform for the initial Trading Cockpit.

Platform-specific behaviour shall be isolated where practical.

Application Architecture

The application follows:

- Clean Architecture
- Domain-Driven Design
- Event-Driven Architecture
- Modular Monolith
- Dependency Injection
- Explicit Dependencies

Primary layers:

1. Domain
2. Application
3. Infrastructure
4. Presentation

Dependency direction shall always point toward the Domain.

Presentation shall not contain business rules.

Infrastructure shall not make trading decisions.

Desktop Application

The Trading Cockpit shall be implemented as a desktop application using PySide6.

The desktop application shall provide:

- deterministic startup
- deterministic shutdown
- responsive user interaction
- centralized runtime lifecycle
- configurable workspaces
- dockable widgets
- persistent workspace state
- visible operational state

Long-running or blocking operations shall not execute on the UI thread.

Workspace Architecture

The workspace is a core product capability.

The workspace shall support:

- dockable widgets
- resizable widgets
- multiple widget instances where appropriate
- persistent layout state
- workspace restoration
- instrument context sharing
- user workflow preservation

Workspace state shall be stored independently from business data.

Invalid workspace state shall not prevent application startup.

Widget Architecture

Widgets shall be reusable presentation components.

Every widget shall:

- have a defined responsibility
- expose explicit dependencies
- remain independently testable where practical
- support application theming
- support workspace integration
- provide visible loading and error states
- avoid direct infrastructure access

Widgets shall interact with application services or presentation adapters.

Widgets shall not contain domain business rules.

Shared Instrument Context

The Trading Cockpit shall support shared instrument context.

Selecting an instrument in one compatible widget may update other context-aware widgets.

Examples:

- Watchlist → Chart
- Scanner → Chart
- Scanner → Decision Center
- Position Overview → Chart

Context propagation shall be explicit and observable.

Widgets shall define whether they:

- follow shared context
- publish shared context
- remain context-independent

Runtime Architecture

The runtime shall manage:

- configuration
- logging
- dependency injection
- infrastructure initialization
- application services
- background workers
- health monitoring
- controlled shutdown

Runtime lifecycle:

1. Bootstrap
2. Load configuration
3. Initialize logging
4. Build dependency container
5. Initialize infrastructure
6. Initialize application services
7. Initialize presentation
8. Enter operational state
9. Controlled shutdown

Application logic shall not execute before initialization is complete.

Asynchronous Processing

AsyncIO shall be used where asynchronous execution is required.

Typical asynchronous operations include:

- broker communication
- market data updates
- background services
- scheduled tasks
- external integrations

Rules:

- never block the event loop
- never block the UI thread
- use explicit timeouts for external operations
- support graceful cancellation
- preserve deterministic shutdown

Async boundaries shall remain explicit.

Market Data Requirements

Market data capabilities shall provide:

- instrument identification
- quote updates
- timestamps
- market state
- data freshness information
- stale data detection
- controlled subscription lifecycle

Market data shall expose its operational state.

The UI shall distinguish between:

- current data
- stale data
- unavailable data
- disconnected state

Trading workflows shall not silently treat stale data as current data.

Broker Integration

Broker integrations shall be implemented through infrastructure adapters.

The initial broker integration may support Interactive Brokers.

Broker adapters shall expose:

- connection state
- account information
- portfolio state
- position state
- order state
- execution information
- broker errors

Domain and application logic shall not depend directly on broker-specific APIs.

Broker-specific identifiers and behaviour shall remain isolated where practical.

Order Safety Requirements

Order workflows are business-critical.

The system shall:

- validate orders before submission
- display order parameters before execution
- prevent uncontrolled duplicate submissions
- expose broker acknowledgement
- expose rejection state
- expose execution state
- preserve order lifecycle history
- reconcile local and broker state

Order submission shall never fail silently.

Unexpected order state shall be visible and logged.

LIVE trading capabilities require explicit product and operational approval.

Portfolio and Position Requirements

Portfolio and position information shall provide:

- account context
- current positions

- quantities
- average prices where available
- current valuation where available
- unrealized P&L; where available
- realized P&L; where available
- data timestamps
- source state

Local state and broker state shall remain distinguishable.

State discrepancies shall be detectable and observable.

The system shall not invent missing financial values.

Risk Requirements

Risk information shall support trading decisions.

Initial risk capabilities may include:

- portfolio exposure
- position exposure
- concentration
- available broker risk information
- margin information where available

Risk data shall expose:

- source
- timestamp
- availability state

Unavailable risk information shall be shown as unavailable rather than estimated silently.

Decision Support Requirements

The Decision Center shall support a transparent trading decision process.

Decision information may include:

- instrument
- candidate state
- market context
- portfolio context
- risk context
- notes
- tags
- signals
- strategy evaluation
- decision status

Decision support shall preserve relevant context.

Future AI-assisted capabilities shall remain distinguishable from deterministic system information.

Data Storage

Primary local storage:

- SQLite

Persistence access shall use:

- Repository Pattern
- SQLAlchemy where appropriate

Storage responsibilities may include:

- application state
- workspace metadata
- watchlists
- trading candidates
- decisions

- local order state
 - journal information
 - configuration references
- Business logic shall not depend directly on SQLite or SQLAlchemy.
-

Configuration

- Configuration shall be:
- externalized
 - explicit
 - validated during startup
 - environment-aware
 - independent from business logic

Primary configuration format:

- YAML

Secrets shall not be stored in source-controlled configuration files.

Configuration precedence shall remain deterministic.

Logging

The application shall use structured logging.

Logging shall provide sufficient context for:

- startup
- shutdown
- configuration
- broker connectivity
- market data state
- order lifecycle
- position reconciliation
- background services
- critical failures

Sensitive information shall never be logged.

Business-critical lifecycle events should remain traceable.

Monitoring and Health

The application shall expose operational health information.

Monitoring may include:

- runtime state
- broker connection state
- market data state
- background worker state
- event processing
- resource usage
- critical failures

Monitoring shall not change application behaviour.

Operational state shall be visible to the user where relevant.

Performance Requirements

The Trading Cockpit shall remain responsive during normal operation.

Requirements:

- UI operations shall not be blocked by external integrations
- background work shall remain isolated from the UI thread
- market data updates shall not cause uncontrolled UI refresh rates

- widget updates should be incremental where practical
 - unnecessary allocations should be avoided in high-frequency paths
 - startup performance shall be measurable
- Performance optimization shall be based on measurement.

Error Handling

Errors shall be:

- explicit
- logged appropriately
- observable where operationally relevant
- isolated where practical

The system shall avoid cascading failures.

Critical failures shall support controlled shutdown.

User-facing errors shall explain the operational impact without exposing internal implementation details unnecessarily.

Security Requirements

The system shall:

- keep secrets outside source control
- validate external input
- isolate external integrations
- use least-privilege principles where practical
- avoid logging sensitive information
- protect broker credentials

Security-relevant changes shall be reviewed explicitly.

Quality Requirements

Production code shall provide:

- full type hints
- Ruff compliance
- automated tests
- deterministic tests
- documented public interfaces
- explicit error handling
- maintainable structure

Critical trading workflows require automated test coverage.

Regression tests shall be added for corrected critical defects.

Testing Requirements

Testing shall include:

- unit tests
- integration tests
- presentation logic tests where practical
- broker adapter tests
- persistence tests
- runtime lifecycle tests

Trading-related workflows shall be validated in PAPER mode before LIVE use.

LIVE validation requires explicit approval.

Documentation Requirements

Markdown under `docs/` is the documentation source of truth.

Generated formats:

- DOCX
- PDF

Generated documents shall be produced through the documentation generation pipeline.

Documentation shall remain synchronized with:

- product behaviour
- architecture
- public APIs
- operational workflows

Technical Evolution Principles

New technical capabilities shall:

- support a defined product workflow
- preserve architecture boundaries
- avoid premature generalization
- remain testable
- remain observable
- remain documented

Prefer vertical product slices over isolated infrastructure expansion.

Technical Review Checklist

Before releasing technical changes verify:

- product workflow identified
- architecture boundaries preserved
- UI thread remains responsive
- async boundaries are explicit
- external state is observable
- error handling is defined
- automated tests added
- documentation updated
- LIVE trading impact evaluated where applicable

Related Documents

- Product_Vision.md
- Product_Roadmap.md
- Project_Overview.md
- Architecture.md
- Domain_Model.md
- Infrastructure.md
- Widget_Catalog.md
- UI_Guidelines.md
- Configuration.md
- Runtime.md
- Logging.md
- Monitoring.md
- Testing_Strategy.md
- Coding_Standards.md
- AGENTS.md